

Subterranean Salvage

Subterranean Salvage (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): A collection-based extraction game in a 2D pixel-art style for the web. The player is a treasure hunter sending drones into the Red Zone, evading enemies with heightened hearing by making minimal sound and sneaking, looting homes and chests for items lost to time, and returning to the safe zones before the drone's limited battery runs out. Collected items grant points (rarer = more), and players compete on a leaderboard. Named technical features: a **sound-based raycasting system** (object interactions produce noise; enemy AI detects and pathfinds to the audio source); a **real-time surface lighting system** (a cone light dynamically illuminates surroundings when a light source is equipped); a **dynamic per-level difficulty framework** (positions enemies around high-value loot and key objectives); and an **online leaderboard API + cloud database**. The load-bearing C++ does real-time acoustic raycasting — radial sound waves on drone noise (fast propulsion, cracking chests) with falloff, obstacle dampening and reflections to alert sound-sensitive enemies. Custom tool: a Red Zone Extraction Level Editor (paint tile layouts, place safe extraction zones, position homes/ruins/loot chests). Design features: environment-driven gameplay; evasion-based extraction (no combat — only outrun/hide/sneak); ≥ 3 sets of 3 collectable items depicting the past from a post-apocalyptic view; ≥ 3 enemy zombie types (Normal, Avoidant, Aggressive) with different interactions/stats.

References:

- [Mark of the Ninja — Wikipedia](#) (2D stealth where sounds are shown as expanding circles that alert enemies, with light/dark visibility and a scoring system)
- [Darkwood — Wikipedia](#) (top-down survival where you scavenge/loot and evade enemies drawn by sound and light)

- [Escape from Tarkov — Wikipedia](#) (the extraction genre: scavenge loot in a raid and reach an extraction zone within the time limit or lose it)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✗ Not needed | » Second trimester

Core Tech Requirement	Subterranean Salvage
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✗
Networking System	●

How each requirement applies:

Platform fit: **Main:** Web browser on mobile — the proposal targets web applications (HTML/JS/CSS + the C++ engine compiled to WASM via Emscripten), touch-first on the phone (device: Pixel 9a); the Native PC build is the development/tools & benchmark vehicle. **Extended:** Firebase + Linux server set up by M2 (full use T2).

Front end: The pixel-art Red Zone renderer (drone, 3 enemy types, homes/ruins/chests, items), the real-time cone-light/surface lighting, sound-

visualisation and detection cues, the battery/loot HUD, safe-zone/extraction UI and the leaderboard (OpenGL ES / WebGL). **Languages:** Web — HTML/JS/CSS + C++→WASM (Emscripten, SDL2/3, OpenGL ES/WebGL). (Web-on-mobile target — no native Android build this trimester; the Native PC build is the dev/tools vehicle.)

Back end: Engine core (fixed-timestep loop, entity system, drone movement & collision, the acoustic raycasting/propagation service, enemy AI & sound-source pathfinding, loot/battery/extraction systems, per-level difficulty placement) + a leaderboard service. **Languages:** C++.

Backend scope: Local now, server-ready (T2) — leaderboard/cloud DB seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the load-bearing C++ is the **acoustic raycasting service** — on drone noise it casts radial sound waves and computes falloff, obstacle dampening and reflections in real time, feeding sound-sensitive enemy AI that pathfinds to the source; the fixed-timestep loop also advances drone movement, the battery timer, loot and the 3 enemy behaviours.
- *Data-driven:* tile layouts, the Red Zone/homes/ruins/chests, the 3 enemy types and stats, the item sets, difficulty rules and acoustic/lighting parameters are all hot-reloadable JSON — no hard-coded levels, enemies or items.
- *2D:* pixel-art sprite/tile rendering of the Red Zone, drone, enemies, items and chests, plus the real-time cone-light/surface lighting and sound-wave visualisation (custom-rendered). *3D:* not needed. *Networking:* async online leaderboard; no real-time netcode.
- *Custom tool:* the **Red Zone Extraction Level Editor** — paint tile layouts, place safe extraction zones and position homes, ruins and loot chests, with live preview.
- *External DB:* the cloud leaderboard/high-score database. *Auth »:* accounts for leaderboard identity. *Analytics »:* optional — where runs fail (detected / battery-out / by which enemy), for difficulty tuning.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in `milestone-goals-pool-per-category.pdf` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window with touch input and a fixed-timestep loop advancing the drone, the battery timer, the acoustic pass and enemy updates; C++17/20, CMake, Git; Emscripten→WASM for the web target.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; M1G6 acoustic raycasting and M1G8 enemy AI run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Entity/component registry in C++ with tiles, the Red Zone/ chests/items, the 3 enemy types & stats, difficulty rules and acoustic/lighting parameters in JSON (nlohmann/json) with hot-reload.
- **Prerequisites:** G1 (loop & entity update hooks).
- **Feeds into:** M2G2 (the level editor edits this data), M1G6/M1G8 (acoustic + enemy data), M2G10 (difficulty placement).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays for sound-wave propagation, enemy alert state and detection, Tracy profiling, Catch2 stress tests (many simultaneous sound sources + enemies), GitHub Actions Windows+Linux matrix.
- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening) and the WASM frame budget M2G1 validates.

- **G6 Audio System** — the acoustic raycasting/propagation service: radial sound waves, falloff, obstacle dampening and reflections that alert enemies.

Tech & dependencies

- **Tech:** A C++ acoustic model casting radial rays on noise events (propulsion, chest-cracking), computing falloff, obstacle dampening and reflections, and emitting alert events to enemy AI; FMOD/OpenAL for the actual SFX playback tied to the same events.
 - **Prerequisites:** G1 (loop/tick), G2 (acoustic parameters as data).
 - **Feeds into:** M1G8 (enemies pathfind to the loudest source); the core stealth mechanic.
- **G8 Enemy AI & Pathfinding** — hearing-based detection + pathfind-to-sound for the 3 enemy types (Normal, Avoidant, Aggressive).

Tech & dependencies

- **Tech:** A*/flow-field pathfinding to the alerting sound source, steering, and finite-state machines for the 3 enemy types' distinct behaviours/stats (custom C++), tuned from data.
 - **Prerequisites:** G1 (tick), G2 (enemy data), G6 (sound alerts).
 - **Feeds into:** M2G10 (advanced behaviour + difficulty placement); the evasion loop.
- **G5 Physics & Collision** — drone movement, collision with the environment/chests, and the raycast geometry the acoustic system uses.

Tech & dependencies

- **Tech:** C++ + GLM math, a custom integrator for drone movement, broadphase (uniform grid / quadtree), collision resolution, and ray/segment intersection against walls/obstacles (shared by acoustics and line-of-sight).
- **Prerequisites:** G1 (loop & tick).
- **Feeds into:** M1G6 (obstacle dampening/reflection geometry); movement feel.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling of the acoustic/AI passes, ASan on MSVC, memory pools for enemies/sound events, and Emscripten→WASM validation on the mobile-web target plus Pixel 9a browser checks; Linux server set up (account + software + basic connectivity test) by M2.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).

- **Feeds into:** The web/mobile deliverable and headroom for dense acoustic/enemy scenes.
- **G2 ★ Content Editor — Core** — the Red Zone Extraction Level Editor: paint tiles, place safe/extraction zones, and position homes, ruins and loot chests.
 - Tech & dependencies
 - **Tech:** ImGui editor authoring tile layouts, extraction/safe zones, structure and chest placement, enemy spawns and item tables as JSON, with round-trip serialization and an undo/redo command stack.
 - **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (live Red Zone preview).
 - **Prerequisites:** G2 (tile/entity data), M1 rendering.
 - **Feeds into:** The playable Red Zone maps; M2G10 (difficulty tuning applied through it).
- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**
 - Tech & dependencies
 - **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, and packaging of the Emscripten web build artifact (and the desktop dev build) via GitHub Actions.
 - **Prerequisites:** G3 (CI and test foundation).
 - **Feeds into:** Release/showcase builds of every Subterranean Salvage goal.
- **G4 2D Graphics — Advanced** — the real-time cone-light and surface lighting system.
 - Tech & dependencies
 - **Tech: Web build:** OpenGL ES / WebGL shaders and framebuffers for a real-time cone light and dynamic surface illumination, with shadow/occlusion from walls; texture atlases for the pixel-art set.
 - **Prerequisites:** M1 2D rendering; M2G1 (optimization).
 - **Feeds into:** The atmosphere and readability of the darkened Red Zone.
- **G10 AI — Advanced** — behavior trees for the 3 enemy types + the dynamic per-level difficulty framework.
 - Tech & dependencies
 - **Tech:** Behavior trees / difficulty tiers over the M1 AI foundation, plus a placement system that positions enemies around high-value loot and key objectives per level (custom C++).

- **Prerequisites:** M1G8 (AI & pathfinding foundation), M1G6 (sound alerts).
- **Feeds into:** Showcase difficulty; the risk/reward around rare loot.
- **G8 Firebase — Accounts, Leaderboards & Analytics** — the online leaderboard + cloud high-score database.
Tech & dependencies
 - **Tech:** Firebase Auth accounts, Firestore leaderboards (points by run), optional Analytics for run-failure funnels, via the REST API from C++ / JS SDK on the web front-end; server-validated score submission.
 - **Prerequisites:** G2 (run/score data model).
 - **Feeds into:** Competitive leaderboards; M2G10 difficulty telemetry.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 2 Art (9) — a 2D pixel-art world (the Red Zone tileset, homes/ruins/chests, the drone, 3 enemy zombie types, ≥ 9 collectable items and the lighting/VFX) is a big art slot, so 2 artists; a solo designer owns the acoustic-stealth/extraction loop, the level and difficulty design and the item/point economy (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If level/content volume grows, trade the 2nd artist for a 2nd designer. *Scope note (advisory):* acoustic sim + real-time lighting + difficulty framework + editor + 3 enemies + 9 items is a large surface — treat the acoustic-stealth extraction core with 1–2 hand-built Red Zones + the editor as the M1/M2 floor, with the difficulty framework and full item sets as the stretch.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the sneak → loot → evade → extract-before-battery loop and the point economy; benchmark e.g. Mark of the Ninja for sound/stealth readability)
- M1D2 Core Gameplay Loop Prototype (greybox proving sound-based evasion + loot + extract is tense and fun) · prototype in Unity
- M1D6 Level & Progression Design (Red Zone layouts, the acoustic/loot risk-reward, and the teach-test-twist curve for the hearing mechanic)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (enemy hearing, battery budget, loot value and difficulty placement via the team's JSON, no code changes)

- **UI/UX M1** (graded by the Design instructor for the Games category):

- M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the battery/loot HUD, the sound-visualisation cue and the extraction prompt.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — standard screens + the loot/extract/leaderboard flow navigable unaided on the mobile-web target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished HUD and touch controls in the browser on the Pixel 9a.

- **Art M1:** 2 artists justified (pixel-art environment + enemies + items + lighting)

- M1A1 ★ (mockups: main menu + a Red Zone in play, lit by the cone light)
- M1A2 Characters (the drone + the 3 enemy zombie types — Normal, Avoidant, Aggressive, silhouette-first)
- M1A3 Environment (Red Zone tileset — homes, ruins, chests, safe/extraction zones)
- M1A4 UI Art Concepts & Typography (battery/loot HUD, sound cue, extraction prompt)
- M1A5 Animation Foundations (drone move/propulsion, enemy idle/alert/chase, chest-open)
- M1A6 Early Asset Production (incl. the first item sets)

- **Art M2:**

- M2A1 ★
- M2A2 Extensive Animation (enemy behaviours per type, drone states, loot interactions)
- M2A3 Environment Alive (ambient motion, the ≥ 9 collectable-item art depicting the past)
- M2A4 UI Art — States & Feedback (heard/alerted, low battery, item rarity, extraction success)
- M2A5 VFX — Effects & Transitions (sound-wave visuals, cone-light glow, extraction)
- M2A6 Cutscenes & Storyboards (intro to the Red Zone premise)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — **audio is load-bearing here** (sound IS the mechanic): the SFX the acoustic system emits (propulsion, chest-crack, footstep, enemy alert) must be present, suitable, and **correctly synced** to the sound-raycasting events in the prototype, plus atmospheric BGM.

- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback (many overlapping sound sources) tied to the acoustic model.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor with a **load-bearing sync clause**: BGM + a full SFX set (propulsion, loot, footsteps, enemy alerts, extraction, ≥6 types) that accurately reflects the acoustic simulation (louder actions = larger, more audible sound).
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback the build needs (multi-channel mixing; spatialisation matching the sound-raycasting so the player can judge how loud they are).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the Red Zone level editor), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*